

Speknife software documentation

version 2026

1 Introduction

Speknife is a software for X-ray spectral correction based on the Stripping method. It was developed in Python 3 and requires no prior Python knowledge from the user. While it is primarily implemented for CdTe spectrometers, it can be easily modified to work with other spectrometer materials. The package includes two folders:

1. *Speknife_Colab*: This folder is intended for users who prefer to use Google Colab. Users must upload the entire *Speknife_Colab* folder to their Google Drive and then open the provided Colab notebook (`speknife.ipynb`) to run the corrections. This adaptation of Speknife was created to be user-friendly for individuals who are not familiar with Python, are intimidated by programming, or do not want or cannot install Python 3 on their machine.

In this version, users will not interact directly with programming code. They only need to modify specific parameters in a text file and click on three buttons to execute the corrections. Detailed instructions are available in the documentation.

Important: The *Speknife_Colab* folder must not be placed inside other folders in Google Drive; otherwise, the program will not run!

2. *Speknife*: This folder is for users who prefer to run the program locally on their computer. In this case, Python 3 must be installed on the system. This version is designed for users who are comfortable with programming. However, even in this version, no prior knowledge of Python is required; the user only needs to edit an input text file to adjust parameters.

2 Requirements

The Google Colab version has no requirements, but you need a Google account to have access to Google Drive and Colab. Local use on your own machine requires Python 3, Matplotlib only for plot generation, NumPy, and SciPy.

3 Principle of operation

Speknife works based on the Stripping method [2, 3]. So it makes corrections in a loop process from the higher energy channel to channel zero. At each channel, it first applies the function to correct escape fractions, then the function that corrects the Compton Plateau, and then the efficiency correction, as Figure 1 shows.

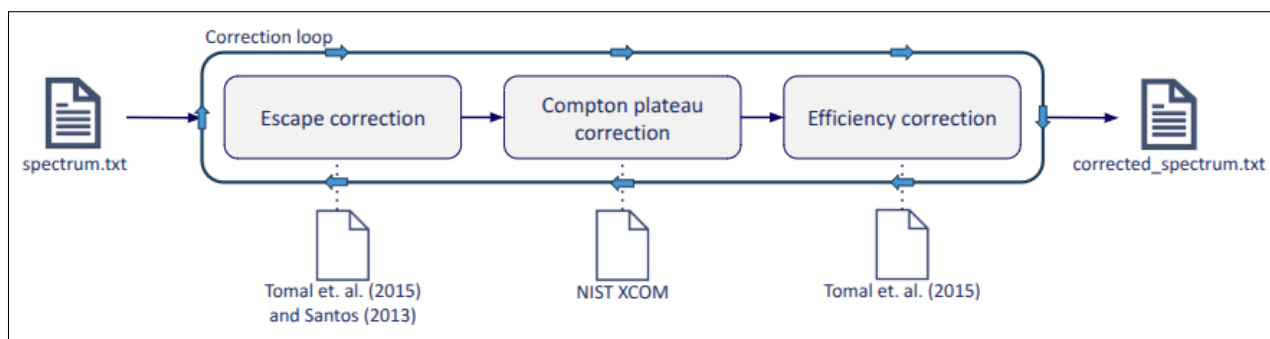


Figure 1: Stripping correction method.

The files in Figure 1 are text files that contain the data necessary to do the correction. In the Speknife software, those files are inside the folder `theoretical_functions_data`.

4 Explaining input file

The input file has a simple structure. On the left, there are the variables and on the right their respective values, as can be seen in Figure 2. The variable names are case-sensitive, so you must write them as in the example file, without uppercase letters.

```

1 #Basic requirements
2 tube_voltage           30 #Must be an integer
3 channel_number         2048
4 a                       0.0303425
5 a_uncertainty           0.0000029
6 b                       -0.0817
7 b_uncertainty           0.0054
8 r_pearson               -0.9826
9 m                       3 #number of data points used in calibration
10 mca                    True
11 uncertainty_analysis    True
12
13 #Plot control
14 plot                    False #show or not the plot
15 plot_type               2
16 normalization_number    1 #plot_type 4 only a normalization 0 is possible, it will ignore any other normalization number you may require
17 save_plot               False
18
19 #Tube voltage analysis
20 tube_voltage_plot        True
21 tube_voltage_measurement True
22 shift                    -6 #kv plot shift
23
24 #Display comparisons plot
25 other_database           False
26 base_name                N
27 reference_path            reference_radiation/PTB_spectra/narrow_beam/ #Path to the reference data to be compared
28 reference_name            PTB #Reference name for the quality to display on the plo

```

Figure 2: Input file example.

4.1 Basic requirements block

For a simple run, that means only to correct your spectra, you have to provide four mandatory arguments. The coefficients a and b of the straight line calibration fit, assuming an equation $y = ax + b$; the tube voltage, which is a nominal value and must be an integer; and the total number of channels. As could be noted, the symbols and names are intuitive. If you don't specify these four arguments, the code will crash.

The other variables are optional and disabled by default. They are there for a more comprehensive use of Speknife. Just one of them, the *mca*, requires more attention. It is not mandatory, but if not specified, it will be set to *False*.

This variable is related to the format of the raw spectra file, your input spectra to be corrected. If *mca* is *True*, the format of the file expected by the code is *file.mca*, that came from Amptek spectrometers. If it is *False*, the code will expect a text file with two columns, the left one with energy values and the right one with the counts separated by spaces, or a one-column file that contains only the data to be corrected.

For an uncertainty analysis on the correction, you must set the variable **uncertainty_analysis** to *True*, by default it is *False*, and then give the values of the *a* and *b* uncertainty and the correlation coefficient, **r_pearson** between *a* and *b*. The **m** value is the number of data points used in the calibration fit, and this will be used for the calculation of the coverage factor of the uncertainties.

4.2 Plot control block

The plot control block has the variable **plot** that, by default, is *False*, and it is to inform the code whether you want to have plots of the correction or not. You also have to provide the type of plot you want to display, considering the five available types to the variable **plot_type**.

0. Only spectra not corrected by the user, raw spectra.
1. Comparison between raw and corrected spectra.
2. Only corrected spectra.
3. Corrected spectra scatter plot with each data point with its respective uncertainty.
4. Corrected spectra, raw spectra, and the contributions of each physical correction on the spectra.

Together with the type of the plot, you need to specify the normalization you want for the data being exhibited. By default, normalization 0 is assumed, and for plot type 4 only normalization 0 is possible.

0. Data is not normalized.
1. Data is normalized by the maximum value.
2. Data is normalized by the total of counts in the spectra.

The last plot option is **save_plot**, *True* by default, this means that when you require plots from the code, they will be saved automatically in the Speknife root folder. If you don't want to save the plot, set the *False* logical value.

4.3 Tube voltage analysis block

This block has only three variables that basically contain whether you want the peak voltage determination, `tube_voltage_measurements`, and whether you want the plot of the straight line fit of the last 15 points of the spectra used for this determination. Both variables are *False* by default.

The `shift` variable has a simple meaning: it is an empirical value you must attribute to adjust the straight line fit, it is 0 by default. The need for this variable is because not always the straight line will pass through the $y=0$ exactly at the nominal tube voltage, so this parameter allows you to shift how many channels you want for the left, negative numbers, and for the right, positive numbers. By shifting the number of the final channel, you will find, by trial and error, the best fit.

4.4 Display comparison plot block

These block options are related to the plots and do not apply with option `plot_type = 4`. These variables allow you to compare your spectrum with reference spectra using any of the four available plots. By default, Speknife has the narrow beam spectra from the ISO 4037:2019 standard from PTB [1]. If you have reference spectra from RQR qualiteis, or whatever you want, it is possible to add them inside Speknife and look at their shape in comparison with your current one.

When the variable `other_database` is `True`, the code will look for a spectra database to display together with the different types of plot mentioned before. However, to have this option working correctly, you need to specify the `base_name` of the spectra you have for the comparisons. This is the prefix name the code will search for the reference spectra you add to compare with your current ones.

Make sure they all have the same prefix name accompanied by their tube voltage, because that is the way Speknife can find the database spectra and place them together in the plots with your measured one. For instance, if you have the `RQR-7.txt` file, modify its name to `RQR-90.txt` (90 is the nominal tube voltage), so that both spectra will appear together in the plot. For this case `base_name` will be `RQR-`.

The `reference_path` is the path where the reference spectra files are placed, and the `reference_name` is just the name you want the spectra to have in the plot label.

5 Folders structures

In Figure 3 there is the Speknife folder structure, which is the same for Colab and local use. As a user, you just need to edit the `input.txt` file and place your own spectra to be corrected inside the working area folder, as the example spectra already available there. When finishing the correction, you should move the files to the repository folder to avoid overriding the files.

If you want to change the Compton and efficiency data, you must modify the files inside the `theoretical_function_data` folder and then change the corresponding filename in `detection_physics.py`.

If you want to compare your corrected spectra with other databases besides the provided one, just create a new folder inside reference radiation and place them there. Just remember to follow the instructions mentioned in the input file description in subsection 4.4 for a corrected selection of the data.

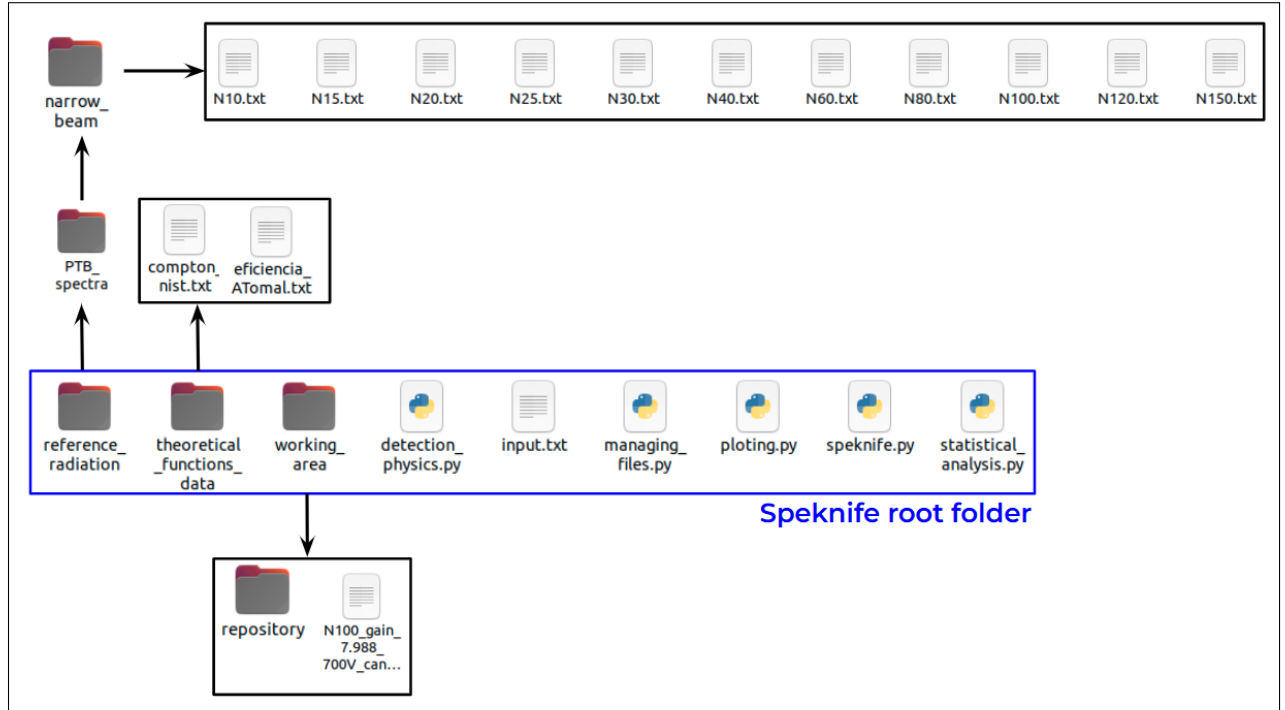


Figure 3: Speknife folder structure.

6 Flowchart of Speknife

The code is organized around a central file called *speknife.py* (*Speknife Core*), which serves as the main engine of the spectrum correction process. This file contains the logic for reading the configuration file (*input.txt*); calling the three functions from the module that implements the physical principles of spectral correction and the uncertainty evaluation associated with the correction operations (*detection_physics.py*); calling the module that contains the functions for uncertainty evaluation of the raw spectrum and the quantities of interest, as well as the estimation of the X-ray peak voltage and the calculation of the mean energy (*statistical_analysis.py*); and calling the module responsible for file input and output operations (*managing_files.py*).

The uncertainty evaluation is continuously performed throughout the software execution. The code reads from the *input.txt* file the parameters required to initiate the calculation based on the calibration curve provided by the user and generates two uncertainty arrays: one for energy and another for counts. The energy uncertainty array is calculated in *speknife.py* using the *spectra_uncertainty* function from the *statistical_analysis.py* module. Likewise, the count uncertainty array is initially calculated by the same function; however, it is subsequently updated after each arithmetic operation performed during the spectral correction process in the *detection_physics.py* module.

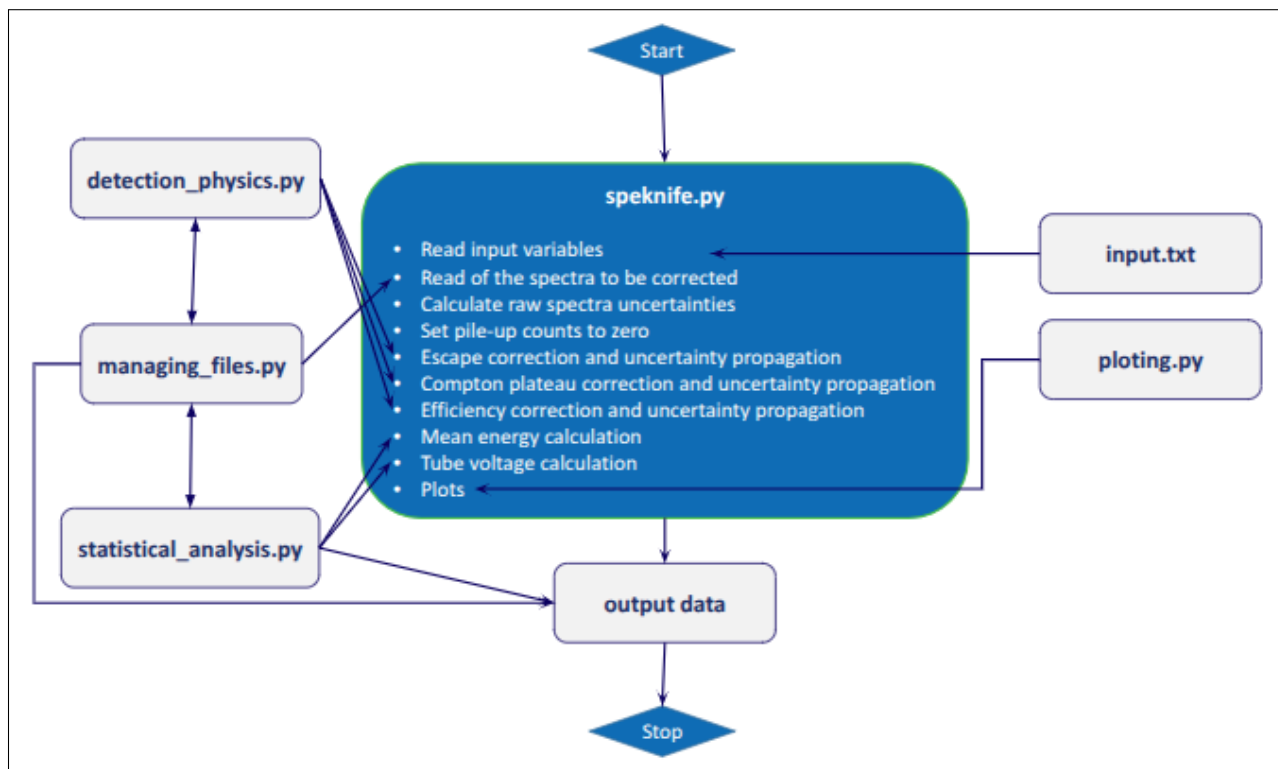


Figure 4: Speknife flowchart

7 How to cite Speknife

Please, when using Speknife for your corrections, cite this work that explains the base theory <https://doi.org/10.1002/xrs.70119>.

8 Google Colab version

The variables and process described here in the documentation apply to the Colab version, the only difference is that it is simpler to use, and you only click three buttons to do the corrections. Just follow the instructions there, and you will be fine. The Colab file image can be seen in Figure 5.

To run the correction, you need to:

- Click the Run button for the Drive mount,
- Run the input.txt configuration
- Finally, click the Speknife button.

For full access to the Python code, it is recommended to open the developer version. You can contribute by making your own improvement suggestions on GitHub (<https://github.com/matheusrebe/Speknife>).

```
[ ] #Drive mount button
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[ ] #Input.txt button for simple correction
%%writefile input.txt

#Basic requirements
tube_voltage      100 #Must be an integer
channel_number    2048
a                 0.07599
b                 -0.08387
mca               True

#Plot control
plot              True #show or not the plot
plot_type         2
normalization_number 1 #plot_type 4 only a normalization 0 is possible, it will ignore any other normalization number you may require
save_plot         True
```

Writing input.txt

```
#Speknife button
import sys
sys.path.append('/content/drive/MyDrive/Speknife_Colab')

import speknife as spk
```

Figure 5: Google Colab version.

9 Collaborators

Matheus Rebello do Nascimento, Luís Fernando de Oliveira, José Guilherme Pereira Peixoto, Luís Alexandre Gonçalves Magalhães, Luis Vicente Gomes Tarelho.

References

- [1] Ulrike Ankerhold. *Catalogue of X-ray spectra and their characteristic data : ISO and DIN radiation qualities, therapy and diagnostic radiation qualities, unfiltered X-ray spectra*. Physikalisch-Technische Bundesanstalt (PTB), 2013.
- [2] E Di Castro, R Panit, R Pellegrinit, and C Baccis. The use of cadmium telluride detectors for the qualitative analysis of diagnostic x-ray spectra. *Phys. Med. Biol*, 29(9):1117–1131, 1984.
- [3] Matheus Rebello do Nascimento, Josilene Cerqueria Santos, Eric Matos Macedo, Leonardo de Castro Pacífico, and Luís Alexandre Gonçalves Magalhães. The stripping method for x-ray spectral correction in iso 4037-1 narrow spectra series (n10-n150). *X-Ray Spectrometry*, n/a(n/a), 2026.